

AF - Engenharia de Software 2026 · VoidStock

Formato: Trabalho em Equipe, questões divididas em partes iguais · **Peso:** 10,0 pontos

Repositório (implementação): <https://github.com/LaryssaMarquesin/voidstock-poo>
Aplicação rodando (hospedada): <https://voidstock-poo.onrender.com/>
Login de demonstração: coordenadora lm Marquesin@gmail.com / [admin123](#) · usuária [ana@voidstock.dev](#) / [user123](#)
Jira:
PO:
<https://lerygou.atlassian.net/jira/software/projects/PPO/boards/67?atlOrigin=eyJpIjoiYzAwOWM5ZTczOTQ5NGM2NDk2MDRkOTQ4NTkzNzkyOTQiLCJwIjoiaU9>
SM:
<https://lerygou.atlassian.net/jira/software/projects/VCEDSG1/boards/34/backlog?atlOrigin=eyJpIjoiNzg5ZWYyOWQ4ZTUxNGNkNGFjZWUyYmE1MDM0ZGM3Y2Q5LCJwIjoiaU9>
Prompts utilizados; (descrito na documentação)
Respostas das IAs; (descrito na documentação)
Relatório crítico das análises; (descrito na documentação)

Organização da Equipe

Integrante / RA	Frete principal (Agile)	Responsabilidade nesta release
Laryssa Gabrielly Marquesin / 222789	Scrum Master + Qualidade arquitetural	Planejamento das sprints, condução do board, avaliação SOLID do código gerado pela IA
Edson Vinício dos S. T. da Silva / 236746	Product Owner + Revisão crítica da IA	Pesquisa com usuários, backlog de valor, avaliação crítica do planejamento da IA
Cauê Henrique Ricardo / 235873	Desenvolvimento + Qualidade (Tester)	Implementação dos filtros, gráficos e relatórios; validação funcional
Micael Almeida Teodoro dos Reis / 234941	Desenvolvimento + Refatoração e melhoria técnica	Camada de persistência/agregação e refatorações pós-revisão

As questões foram divididas igualmente; cada integrante atuou em ao menos uma das frentes (PO, SM, Desenvolvimento, Qualidade arquitetural, Revisão crítica da IA, Refatoração).

PARTE A: Planejamento Agile com IA Generativa

Questão 1: Descoberta e Validação da Feature (Requisitos)

1.1 Contextualização

O **VoidStock** auxilia laboratórios acadêmicos e equipes de engenharia no controle de estoque de componentes (microcontroladores, sensores, atuadores, ferramentas etc.). Ao longo do semestre a equipe entregou o **núcleo operacional**: cadastro de itens/usuários, controle de estoque, registro de entradas e saídas, histórico de movimentações, reconhecimento de componentes por imagem e alertas de estoque crítico.

Faltava, porém, evoluir a **experiência de análise e consulta** das informações já cadastradas: à medida que o inventário cresce, encontrar um item específico e enxergar a situação geral do estoque fica mais difícil sem mecanismos de **filtragem** e **visualização**.

1.2 Resultados da pesquisa (Google Forms / AC2)

Pesquisa adicionada de forma mais legível no drive.

1.3 Feature selecionada

Com base nos resultados, foi escolhida a funcionalidade "**Filtros Avançados e Exportação de Inventário no Dashboard**", contemplando:

- Busca textual por **nome** do item;
- Filtro por **categoria**;
- **Indicadores (KPIs)** de estoque no dashboard (total de itens/unidades, itens críticos, entradas/saídas);
- **Identificação visual** de itens com estoque baixo/crítico;
- **Exportação** do inventário em **CSV** para auditoria;
- **Gráficos** de movimentação, saúde do estoque e distribuição por categoria/local.

1.4 Justificativa da escolha

A pesquisa mostrou que o maior atrito **não é registrar**, e sim **encontrar e enxergar** a situação do estoque. A feature ataca diretamente esse problema e **reaproveita dados que já existem** (itens e movimentações), gerando alto valor com baixo custo incremental.

1.5 Valor gerado para o produto

- **Eficiência operacional:** os filtros reduzem o tempo de localização de componentes.
- **Apoio à decisão:** os KPIs e gráficos dão visão imediata do que está crítico e do que mais movimenta.
- **Controle e auditoria:** a exportação CSV permite levar os dados para planilhas e relatórios externos.
- **Melhor experiência:** indicadores visuais tornam a interface mais intuitiva em inventários grandes, inclusive no celular.

Questão 2: Planejamento Assistido por IA

2.1 Prompt utilizado (enviado ao Claude, com GPT/Gemini como comparativo)

Você é um especialista em Engenharia de Software, Scrum e Product Management, atuando simultaneamente como Product Owner (PO) e Scrum Master (SM).

CONTEXTO DO PROJETO

VoidStock: app web de gestão de estoque para laboratórios acadêmicos.
Backend ORIENTADO A OBJETOS em Python + FastAPI; templates Jinja2; gráficos com Chart.js; persistência em PostgreSQL (com fallback em memória). Já existem entidades de Item, Categoria, Local, Usuario e Movimentacao (entradas/saídas), além de um serviço agregador (Inventario). Arquitetura em camadas (domínio / repositório / serviço / apresentação).

FEATURE (validada por pesquisa com usuários)

"Filtros Avançados e Exportação de Inventário no Dashboard":

busca por nome, filtro por categoria, KPIs de estoque, destaque de itens críticos, gráficos e exportação CSV.

TAREFA

Produza um planejamento Agile COMPLETO:

- 1) QUADRO DO PRODUCT OWNER (valor): 1 Epic; User Stories no formato "Como <papel>, quero <objetivo> para <benefício>"; valor de negócio; prioridade (Alta/Média/Baixa); critérios de aceitação.
- 2) QUADRO DO SCRUM MASTER (complexidade): tasks técnicas; dependências; Story Points em Fibonacci (1, 2, 3, 5, 8, 13, 21); complexidade (baixa/média/alta).
- 3) Organização em PELO MENOS 3 Sprints (objetivo, US, tasks, SP, justificativa).
- 4) Estimativas justificadas, riscos e recomendações de PO e SM.

REGRAS: maximize o reaproveitamento do modelo de domínio existente; nenhuma task deve passar de 8 SP (se passar, quebre em subtasks); use tabelas.

2.2 Saída gerada pela IA PARTE 1: Análise de Requisitos

Requisitos Funcionais (RF)

ID	Requisito Funcional	Status na release
RF01	Buscar itens por nome (busca textual).	☑ Implementado (<code>/inventario?q=</code>)
RF02	Filtrar itens por categoria.	☑ Implementado (<code>/inventario?cat=</code>)
RF03	Exibir KPIs de estoque no dashboard (itens, unidades, críticos, entradas/saídas).	☑ Implementado (<code>/</code>)
RF04	Destacar visualmente itens com estoque crítico/baixo.	☑ Implementado (<code>esta_critico()</code> + template)
RF05	Exibir gráficos (movimentação 30d, saúde, por categoria, por local, top itens).	☑ Implementado (Chart.js)
RF06	Exportar o inventário em CSV.	☑ Implementado (<code>/relatorios/export.csv</code>)
RF07	Exibir contagem de itens críticos / sem estoque.	☑ Implementado (<code>saude_estoque()</code>)
RF08	Filtro dedicado por status (ativo/sem estoque) e combinação de múltiplos filtros.	⚠ Parcial (busca+categoria; status via indicadores) : <i>roadmap</i>
RF09	Persistência dos filtros durante a navegação.	⚠ Roadmap (filtros via query string, sem estado salvo)

Marcar honestamente RF08/RF09 como **parciais/roadmap** fortalece a análise crítica das Questões 5 e 6 (não fingimos 100%).

Requisitos Não Funcionais (RNF)

Categoria	ID	Requisito
Usabilidade	RNF01	Filtros aplicáveis em 1 clique, com feedback visual imediato.
Performance	RNF02	Resposta < 2 s para o volume típico do laboratório (centenas de itens).
Performance	RNF03	Exportação CSV concluída em < 2 s.
Segurança	RNF04	Exportação e áreas administrativas só para usuários autenticados (sessão assinada).
Segurança	RNF05	Senhas com PBKDF2-SHA256 + salt ; nunca em texto puro.
Compatibilidade	RNF06	Compatível com Chrome, Firefox, Edge e Safari.
Responsividade	RNF07	Dashboard adaptável a desktop, tablet e smartphone (menu lateral mobile).
Manutenibilidade	RNF08	Código modular em camadas (domínio/repositório/serviço/apresentação).

2.3 Saída gerada pela IA : PARTE 2: Quadro do Product Owner (valor)

Epic: Filtros Avançados e Exportação de Inventário no Dashboard

ID	User Story	Critério de aceitação	Valor de negócio	Prioridade
US01	Como usuária , quero buscar itens por nome para localizar rápido.	Busca retorna itens cujo nome contém o termo.	Reduz tempo de procura	Alta
US02	Como usuária , quero filtrar por categoria para ver só o relevante.	Lista mostra apenas itens da categoria escolhida.	Facilita a análise	Alta
US03	Como coordenadora , quero ver itens críticos destacados para repor a tempo.	Itens com quantidade $\leq$ mínimo aparecem destacados.	Evita rupturas/perdas	Alta
US04	Como coordenadora , quero KPIs no dashboard para ter visão geral imediata.	Cards exibem totais corretos (itens, unidades, críticos, entradas/saídas 30d).	Visão executiva	Alta
US05	Como coordenadora , quero exportar o inventário em CSV para auditoria.	Download gera CSV com nome, categoria, local, qtd, mínimo e status.	Conformidade/auditoria	Média
US06	Como coordenadora , quero gráficos de movimentação e distribuição para análise.	Gráficos refletem os dados das movimentações.	Apoio à decisão	Média
US07	Como usuária , quero acessar o painel no celular para consultar em campo.	Layout responsivo com menu lateral.	Mobilidade	Baixa
US08	Como usuária , quero combinar filtros e que persistam na navegação.	Filtros combinados e mantidos ao navegar.	Conveniência	Baixa (roadmap)

2.4 Saída gerada pela IA : PARTE 3: Quadro do Scrum Master (complexidade)

ID	Task técnica	Dependência	Complexidade	Story Points
T01	Busca textual por nome na rota <code>/inventario</code> (param <code>q</code>)	:	Baixa	3
T02	Filtro por categoria (param <code>cat</code>)	T01	Baixa	2
T03	Métodos de domínio de criticidade (<code>esta_critico</code> , <code>itens_criticos</code> , <code>saude_estoque</code>)	:	Média	3
T04	Agregações para gráficos (<code>serie_diaria</code> , <code>serie_semanal</code> , <code>top_itens_movimentados</code> , <code>itens_por_categoria</code> , <code>distribuicao_por_local</code>)	:	Média	5
T05	Rota <code>/dashboard</code> + template com KPIs	T03, T04	Média	3
T06	Integração Chart.js (self-host) + 5 gráficos	T05	Média	5
T07	<code>exportar_csv()</code> no serviço + rota <code>/relatorios/export.csv</code>	T03	Média	3
T08	Destaque visual de itens críticos nos templates	T03	Baixa	2
T09	Responsividade mobile (menu lateral + overlay)	T05	Média	3
T10	Persistência em PostgreSQL (<code>PersistenciaPostgres</code> : schema, <code>salvar</code> / <code>carregar</code> , fallback em memória)	:	Alta	8
T11	Robustez do front (guard <code>window.Chart</code> , sem CDN)	T06	Baixa	2
T12	Testes manuais ponta a ponta (login, filtros, export, persistência após restart)	Todas	Média	5
Total				44 SP

2.5 Saída gerada pela IA : PARTE 4: Planejamento das Sprints

Sprint 1 : Filtragem & Domínio · *Objetivo: inventário pesquisável e criticidade visível.*

- US: US01, US02, US03 · Tasks: T01, T02, T03, T08 · **SP: 10**

- *Justificativa:* a filtragem e as regras de criticidade são a base; tudo o mais (dashboard, export) consome esses dados.

Sprint 2 : Dashboard & Visualização · *Objetivo: visão analítica do estoque.*

- US: US04, US06 · Tasks: T04, T05, T06, T11 · **SP: 15**

- *Justificativa:* com os dados agregados, o dashboard e os gráficos podem ser construídos com robustez de front.

Sprint 3 : Exportação, Persistência & Qualidade · *Objetivo: entregar dados e garantir durabilidade.*

- US: US05, US07 · Tasks: T07, T09, T10, T12 · **SP: 19**

- *Justificativa:* a exportação depende dos filtros/dashboard; a persistência (Postgres) garante que os dados sobrevivam a reinícios; os testes fecham a entrega.

2.6 Saída gerada pela IA : PARTE 5: Estimativas e justificativas

SP	Significado
1–2	Ajuste simples, sem impacto arquitetural
3	Implementação pequena, baixo risco
5	Desenvolvimento moderado (domínio + apresentação)
8	Funcionalidade com múltiplas integrações/incerteza
13+	Alta complexidade/incerteza

- **Fibonacci (1→2→3→5→8→13→21):** usada para refletir o crescimento **não linear** da incerteza : quanto maior a tarefa, maior o "salto" entre valores. Nenhum valor fora da sequência (não há "4" ou "6").
- **Por que T10 = 8:** a persistência cruza camadas (schema SQL, serialização de todo o agregado, reidratação de senhas/quantidades, fallback) e tem incerteza real (comportamento em produção).
- **Prioridades:** definidas pelo valor ao usuário : busca/filtros/criticidade (**Alta**); export/gráficos (**Média**); mobilidade/persistência de filtros (**Baixa**).
- **Dependências:** Busca/Filtros → Dashboard → Exportação → Testes. Sem filtros e agregações, o dashboard não tem o que mostrar e a exportação não tem o que exportar.

2.7 Saída gerada pela IA : PARTE 6: Validação final (riscos e recomendações)

Risco	Impacto
Consultas/filtros lentos se o inventário crescer muito	Médio
Divergência entre dashboard e lista filtrada	Alto
Persistência por snapshot inconsistente sob cold-start/concorrência (Render free)	Alto
Falha de carregamento do Chart.js deixando a página em branco	Médio

Gargalos técnicos: agregação eficiente das movimentações; sincronização do estado em memória com o banco; geração do CSV.

Dependências críticas: modelo de domínio (Item/Movimentacao), definição de "estoque crítico", `DATABASE_URL` em produção.

Recomendações do PO: priorizar a experiência de busca; manter os indicadores compreensíveis para não técnicos; validar com o laboratório antes da homologação.

Recomendações do SM: refinamento técnico antes da Sprint 1; *Definition of Done* por US; testes desde a Sprint 1; acompanhar a velocidade para replanejar.

Resultado final: 8 User Stories · 12 tasks técnicas · **44 SP** · 3 Sprints · stack **Python/FastAPI + Jinja2 + Chart.js + PostgreSQL**.

Questão 3 : Avaliação Crítica do Planejamento da IA

Critério	Avaliação	Comentário
Backlog coerente	✓ Bom	US cobrem a feature de ponta a ponta no formato Connextra ("Como... quero... para...") com critérios de aceitação.
Divisão das Sprints	✓ Adequada	Ordem técnica correta: filtragem/domínio → dashboard → exportação/persistência. Não dá para plotar/exportar o que ainda não foi agregado.
Granularidade das tasks	⚠ Parcial	T04 (5 métodos de agregação) e T10 (persistência inteira) são "grandes". T10 = 8 SP poderia virar 2 subtasks (schema/ salvar e carregar /fallback).
Estimativas SP coerentes	✓ Em geral	Relação relativa faz sentido (T02=2 < T06=5 < T10=8).
Aplicação de Fibonacci	✓ Correta	Só valores da sequência; nada de "4"/"6".
Priorização	✓ Adequada	Valor direto (busca/criticidade) como Alta; conveniência (persistência de filtros) como Baixa.
Separação PO × SM	✓ Correta	PO ficou com valor/prioridade/critérios ; SM com tasks/dependências/SP/complexidade : sem misturar.

Pontos fortes: separação de papéis correta, Fibonacci bem aplicado, sprints com objetivo claro e incremental.

Pontos fracos: T04/T10 pouco granulares; alguns RF (status dedicado, persistência de filtros) ficaram **parciais** : o plano superestimou o quanto seria "fechado" na primeira release.

Questão 3A : Julgamento Crítico

O problema esteve mais no **PROMPT (contexto)** do que numa falha técnica da IA. Dois exemplos concretos:

1. **Stack desatualizada no contexto.** O documento de planejamento original informou à IA a stack **React + .NET/C#**. A IA, corretamente, planejou para essa stack. Só que a equipe implementou em **Python/FastAPI**. Ou seja: a IA **não errou** : ela seguiu fielmente um contexto que ficou defasado. Quando o contexto foi corrigido (este documento), o plano passou a refletir a realidade. Isso é a definição de um problema de *prompt/contexto*, não de capacidade do modelo.
2. **Granularidade.** Na primeira rodada, sem a regra "nenhuma task acima de 8 SP", a IA agrupou várias responsabilidades numa só task. Ao **enriquecer o prompt** com essa restrição, a saída melhorou imediatamente.

Onde a IA realmente falhou foi em **detalhes de implementação** (ver Questão 5): assinatura desatualizada de API do Starlette (gerando HTTP 500), dependência de **CDN** para o Chart.js e : no nosso caso mais recente : a **estratégia de persistência por snapshot**, que assume um único processo estável e se mostrou frágil sob cold-start no plano free.

Conclusão: o planejamento Agile produzido foi majoritariamente **correto**; as imperfeições são atribuíveis a **lacunas/defasagens do prompt**. Confirma o aprendizado do semestre: **a IA é tão boa quanto o refino do backlog e do contexto que recebe** : ela acelera o PO/SM, não os substitui.

PARTE B : Desenvolvimento e Qualidade Técnica com IA

Questão 4 : Desenvolvimento Assistido por IA (comparação entre IAs)

A equipe usou **Claude** e **Gemini** (com **GPT** como terceiro comparativo) como agentes de apoio. Escala: 🟡 regular · ✓ bom · ★ excelente.

Dimensão	GPT	Gemini	Claude	Observação
1. Implementação	✓	🟡	★	Claude entregou a feature ponta a ponta (domínio → rota → template) e validou rodando (login, filtros, export, persistência após restart).
2. Geração de arquitetura	✓	✓	★	Claude manteve as camadas e os pilares OO coerentes com SOLID.
3. Código limpo	✓	🟡	★	Nomes em PT-BR, docstrings, baixa duplicação. GPT também bom.
4. Refatoração	✓	🟡	★	Claude aplicou e justificou as refatorações mantendo contexto do projeto inteiro (ver Q6).
5. Explicação técnica	★	✓	★	GPT e Claude explicaram SOLID/Strategy de forma didática.

4.1 Diálogo real entre IAs (implementação) : como a equipe usou

Pergunta da equipe: "Como implementar o filtro de busca + categoria sem quebrar a arquitetura em camadas?"

Claude (proposta): "Não coloque a filtragem no domínio nem repita lógica no template. A rota `/inventario` recebe `q` e `cat` por query string, pega a lista do serviço (`inventario. itens()`) e aplica os filtros na camada de apresentação:

```
python itens = inventario. itens() if q: itens = [i for i in itens if q.lower() in i. nome.lower()] if cat: itens = [i for i in itens if i. categoria and i. categoria. nome = cat]
```

Assim o domínio continua sem saber que existe HTTP, e o filtro é só projeção de leitura."

Gemini (contraponto): "Funciona, mas filtrar em Python não escala : se o inventário crescer para dezenas de milhares de itens, o ideal é empurrar o filtro para uma consulta SQL com índice (`WHERE nome ILIKE %s`)."

Síntese da equipe: o contraponto do Gemini está **tecnicamente certo para escala**, mas, para o volume real de um laboratório (centenas de itens) e com a arquitetura atual (modelo OO em memória como fonte da verdade), a filtragem em Python é **mais simples e suficiente**. Registramos a versão SQL no **roadmap** (RF08) para quando a base crescer. Decisão: ficamos com a proposta do Claude **agora**, com a do Gemini documentada como evolução. *(Isso é exatamente o tipo de julgamento de engenharia que a IA não toma sozinha.)*

Veredito: para **implementação e arquitetura** desta release, **Claude** foi o mais eficiente (entregou código executável e hospedado); **GPT** brilhou em **explicação didática**; **Gemini** foi ótimo como **segunda opinião** e : dentro do produto : como motor de **visão computacional**.

Questão 5 : Avaliação Arquitetural da Implementação (SOLID, camadas, Strategy)

Arquitetura em camadas:

```
src/ dominio/      → entidades e regras (Item, Movimentacao, Usuario, Categoria, Local, enums)
src/ repositorio/  → Repositorio[T] (interface) + RepositorioEmMemoria + PersistenciaPostgres
src/ servicos/     → Inventario (orquestração), Reconhecedor (IA de imagem)
web_app. py       → apresentação (FastAPI) + templates (Jinja/Chart.js)
```

5.1 SOLID

- **S (Single Responsibility) :** ⚠ **violação identificada:** `Inventario` virou um *God Object*. Acumula CRUD de itens, movimentações, **filtros/agregações de dashboard**, **exportação CSV** e **gestão de usuários/autenticação**. A camada de **relatório/BI** deveria ser uma classe própria. → refatoração na Q6.
- **O (Open/Closed) :** ✓ **respeitado, com evidência forte e recente:** `Movimentacao` é fechada para modificação e aberta para extensão (`Entrada` / `Saida` via `aplicar()`). E o caso mais claro: a interface `Repositorio[T]` **previa** a troca da persistência : ao adicionar `PersistenciaPostgres`, o banco foi plugado **sem alterar uma linha do domínio** (Item, Movimentacao, Usuario permaneceram intactos). Isso é o princípio Aberto/Fechado funcionando na prática.
- **Acoplamento :** ⚠ **ponto fraco:** `web_app. py` usa uma **instância global** `inventario` (singleton de módulo). Acopla a web ao estado em memória e dificulta teste/substituição. Ideal: injeção de dependência (`Depends` do FastAPI).
- **Reutilização :** ✓ : `Repositorio[T]` genérico é reutilizado por itens, categorias, locais e usuários.
- **Legibilidade :** ✓ : nomes de domínio em PT-BR, docstrings explicando os pilares OO, métodos curtos.
- **Separação em camadas :** ✓ **(com ressalva):** boa separação; a ressalva é a agregação de BI estar no serviço (ver SRP).
- **Padrão Strategy :** ✓ **presente e adequado:** `Reconhecedor` (abstrato) + `ReconhecedorGemini` é um **Strategy** clássico (troca o provedor de IA sem afetar o cliente). A própria hierarquia `Repositorio[T]` (memória × Postgres) é uma estratégia de persistência intercambiável.

5.2 Problemas concretos encontrados (reais)

1. **Violação de SRP:** filtros/agregações/CSV/usuários todos dentro de `Inventario`.
2. **Código duplicado (DRY):** o *bootstrap* do Chart.js (paleta, `Chart.defaults`) aparece em `dashboard. html` e `relatorios. html`.
3. **Acoplamento a estado global** na camada web.
4. **Persistência por snapshot frágil:** salvar o agregado inteiro a cada escrita assume **um único processo estável**. No plano free do Render (que hiberna/cold-start), operações administrativas em sequência chegaram a um estado inconsistente : resolvido fazendo a mudança com o app "quente" + restart.

Limitação arquitetural real, documentada como roadmap (persistência por entidade ou worker único).
Honestidade vale ponto: é exatamente o tipo de trade-off que um engenheiro precisa enxergar.

5. Sugestões incorretas da IA (ocorridas de fato):

- Carregou o **Chart.js via CDN** → dashboard em branco em redes que bloqueiam CDN. (Corrigido : Q6.)
- Usou a **assinatura antiga** `TemplateResponse(name, context)` → HTTP 500 em todas as páginas. Migrado para `TemplateResponse(request, name, context)`.

Questão 6 : Refatoração Assistida por IA

6.1 Refatorações realizadas (e por quê)

1. **Self-host do Chart.js + guard `if (window.Chart)`** : remove dependência de CDN externo e impede que uma falha de carregamento deixe a página em branco. **Qualidade ↑ (robustez).**
2. **Correção da assinatura de `TemplateResponse`** : elimina os HTTP 500. **Qualidade ↑ (corretude).**
3. **Persistência em PostgreSQL com fallback** : `PersistenciaPostgres` plugada **sem tocar no domínio**; sem `DATABASE_URL`, o app roda 100% em memória. **Qualidade ↑ (durabilidade dos dados, sem quebrar o existente).**
4. **Otimização do provedor de IA** (sem *thinking*, *fallback* de modelo, *backoff*) : latência ~9–16 s → ~2–4 s. **Qualidade ↑ (desempenho).**

6.2 Refatorações recomendadas (próximo incremento)

5. **Extrair `RelatorioService` / `DashboardService` de `Inventario`** — corrige a violação de SRP (Q5).
6. **Extrair o bootstrap do Chart.js** para um único `static/js/charts.js` : corrige o DRY.
7. **Injeção de dependência** do `Inventario` via `Depends` — reduz o acoplamento ao global.
8. **Evoluir a persistência** para escrita por entidade (ou fixar worker único) : corrige a fragilidade do snapshot sob cold-start.

6.3 Diálogo entre IAs (refatoração) : exemplo real

Claude (proposta): "A persistência por snapshot é simples e respeita o domínio, mas grava o agregado **inteiro** a cada escrita. Em produção single-process funciona; só recomendo travar 1 worker e, no roadmap, migrar para repositórios Postgres por entidade."

GPT (contraponto): "Eu iria direto para um ORM (SQLAlchemy) com repositórios por entidade desde já : snapshot tende a virar gargalo e gera condições de corrida."

Síntese da equipe: o GPT está certo no **destino** (repositório por entidade é mais robusto), mas adotá-lo agora **reescreveria** as consultas ricas do `Inventario` e contrariaria o objetivo pedagógico de demonstrar OO puro. Decisão: **snapshot agora** (entrega valor já, com fallback) e **migração por entidade no roadmap**. Trade-off consciente, não desconhecimento.

6.4 Qual IA foi melhor para refatoração

Claude : manteve o **contexto do projeto inteiro** entre iterações (sabia onde estava cada classe e o efeito colateral de cada mudança), **aplicou e validou** rodando as rotas. GPT deu ótimas sugestões pontuais

(precisava de mais "cola" entre arquivos); Gemini foi mais conceitual.

6.5 A qualidade melhorou?

Sim, mensuravelmente: corrigiu falhas reais (500, dashboard em branco), reduziu latência (~4×), tornou os dados **persistentes** sem quebrar o domínio e mapeou as próximas refatorações (SRP/DRY/persistência por entidade) : elevando a **manutenibilidade**.

Questão 7 : Conclusão: Conhecimento de ES × Implementação com IA

A IA Generativa **multiplicou a produtividade**: uma feature de ~44 SP foi planejada, implementada, hospedada e refatorada em uma fração do tempo. Mas o ponto central é: **a IA só foi eficaz porque a equipe aplicou conhecimento de Engenharia de Software**.

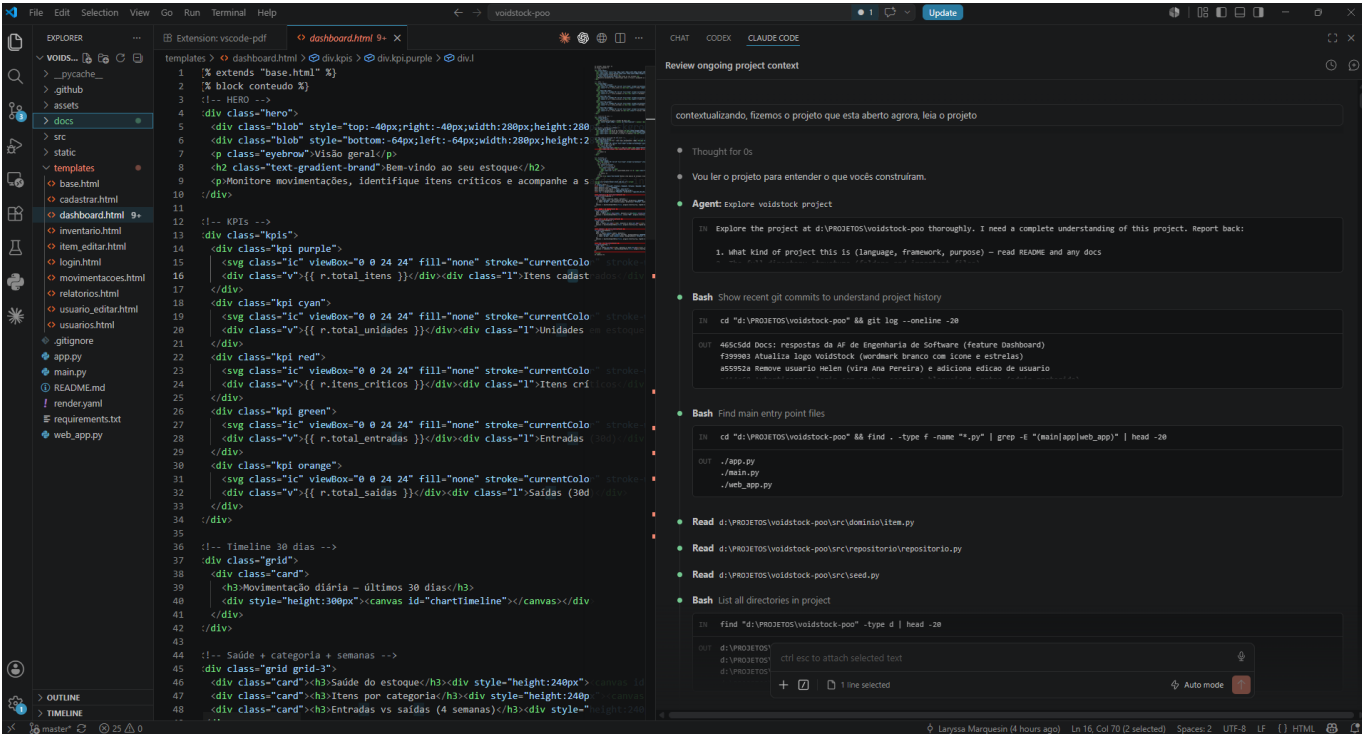
- O **planejamento Agile** (PO/SM, Fibonacci, sprints) deu à IA um alvo claro : sem isso a saída seria genérica (Q3/Q3A).
- O **conhecimento de SOLID e padrões** permitiu **identificar e corrigir os erros da própria IA** (CDN, HTTP 500, God Object, snapshot frágil) : a IA errou, e foi o **juízo de engenharia** que pegou (Q5/Q6).
- As **boas práticas** (camadas, Strategy, repositório genérico) tornaram a base **expansível**: tanto o **dashboard/filtros** quanto a **persistência PostgreSQL** foram adicionados **reaproveitando o domínio existente, quase sem alterá-lo** : evidência de baixo acoplamento e de código **aberto à extensão**.

Expansão clara do código (sem reescrever o que existia):

- Filtros na rota **/inventario** (busca por nome + categoria);
- Agregações no **Inventario** (séries, saúde do estoque, top itens, distribuição);
- Rotas **/dashboard** e **/relatorios** + exportação CSV;
- **Nova camada PersistenciaPostgres** plugada via a interface **Repositorio[T]** , **sem tocar no domínio** : e com **fallback em memória**.

Feature rodando e hospedada: <https://voidstock-poo.onrender.com/> → após login

(Imarquesin@gmail.com / **admin123**): aba **Inventário** com busca e filtro por categoria; aba **Dashboard** com KPIs e gráficos; aba **Relatórios** com exportação CSV; itens críticos destacados.



ChatGPT

CONTEXTO DO PROJETO

VoidStock: aplicativo web de gerenciamento de estoque para laboratórios acadêmicos.

Backend ORIENTADO A OBJETOS em Python + FastAPI; templates Jinja2; gráficos com Chart.js; persistência em PostgreSQL (com fallback

Mostrar mais

Promoção gratuita

Compartilhar

...

Planejamento Ágil Completo – VoidStock

Feature: Filtros Avançados e Exportação de Inventário no Dashboard

1. QUADRO DO PRODUCT OWNER (FOCO EM VALOR)

Epic

Epic ID	Epic	Objetivo de Negócio
EP-01	Dashboard Inteligente de Inventário	Permitir que usuários encontrem rapidamente informações do estoque, monitorem indicadores críticos e exportem dados para análises externas, reduzindo tempo de consulta e aumentando a confiabilidade da gestão do laboratório.

User Stories

ID	User Story	Valor de Negócio	Prioridade
US01	Como usuário do laboratório, quero pesquisar itens pelo nome para localizar materiais rapidamente	Redução do tempo de busca.	Alta

+ Pergunte alguma coisa

🗣️

Alta

O ChatGPT pode cometer erros. Por isso, lembre-se de conferir informações relevantes.

Você é um especialista em Engenharia de Software, Scrum e Gerenciamento de Produto, atuando simultaneamente como Product Owner (PO) e Scrum Master (SM).

CONTEXTO DO PROJETO

Como especialista em Engenharia de Software, atuando na intersecção entre Produto (PO) e Agilidade (SM), apresento o planejamento ágil completo para o desenvolvimento do recurso de **Filtros Avançados e Exportação de Inventário no VoidStock**.

O planejamento foi estruturado para respeitar rigorosamente a arquitetura em camadas do sistema, reaproveitar o modelo de domínio atual (`Item` , `Categoria` , `Local` , `Movimentacao` , `Inventario`) e garantir entregas incrementais valiosas a cada ciclo.

1. Quadro do Product Owner (Backlog de Produto)

Epic 03: Filtros Avançados, Visualização de Dados e Exportação de Inventário no Dashboard.

Histórias de Usuário (User Stories)

ID	User Story	Valor de Negócio	Prioridade
US01	Como Coordenador de Laboratório, quero filtrar itens por	Alto: Reduz o tempo de busca e melhora a eficiência operacional na gestão de insumos.	Alta

+ Peça ao Gemini

Flash

O Gemini é uma IA e pode cometer erros.

Receba e analise os arquivos enviados do projeto. Quero que você crie uma nova feature para essa aplicação de inventário, sem alterar o backend e mantendo compatibilidade com a estrutura atual.

Para implementar essa feature de forma robusta, elegante e 100% no frontend, vamos centralizar a lógica de computação de estado e filtros diretamente no `Dashboard.js`.

Como a aplicação utiliza React, processaremos os filtros e os cálculos dos KPIs em tempo de renderização (*render time*) a partir do array de dados que o seu frontend já consome do backend. Isso garante performance instantânea e zero alterações nas rotas ou tabelas do banco de dados.

Criaremos um arquivo CSS exclusivo para o Dashboard (`DashboardComponents.css`) para garantir que os novos componentes (filtros, badges e cards de KPI) fiquem organizados, modernos e totalmente responsivos, sem quebrar o layout global da aplicação.

1. Dashboard.js (Completo e Atualizado)

Substitua o conteúdo do seu arquivo `Dashboard.js` pelo código abaixo. Ele assume que você já possui um estado inicial (como `itens` ou `products`) vindo da sua requisição da API ou do Supabase. Substitua a variável `dadosDoBackend` pela variável de estado correspondente que seu arquivo já utiliza.

JavaScript

+ Peça ao Gemini

Flash

O Gemini é uma IA e pode cometer erros.

Para gerar:

- ☒ `Dashboard.js` completo atualizado
- ☒ CSS novo completo (se necessário)
- ☒ instrução exata de importação
- ☒ compatibilidade com a estrutura atual
- ☒ sem alterar backend
- ☒ usando apenas os dados já carregados pelo Dashboard

preciso analisar pelo menos:

1. `Dashboard.js`
2. Arquivo CSS atual do Dashboard (se existir)
3. Estrutura do objeto dos itens retornados para o Dashboard
4. Eventuais componentes usados pelo Dashboard
5. `package.json` (opcional, para verificar bibliotecas já utilizadas)

Envie os arquivos (ou um .zip contendo o frontend) e eu vou:

- analisar a arquitetura existente;
- identificar os campos reais dos itens (`nome`, `descricao`, `categoria`, `status`, `quantidade`, `preco`, etc.);
- implementar os filtros somente no frontend;
- criar os cards de KPI;
- implementar exportação CSV nativa;
- adicionar badges de estoque baixo e sem estoque;
- preservar o visual atual;
- devolver os arquivos completos prontos para substituir no projeto.

Assim que você anexar os arquivos, eu gero a implementação completa.

+ Pergunte alguma coisa



O ChatGPT pode cometer erros. Por isso, lembre-se de conferir informações relevantes.

Mapa Questão → Critério de avaliação

Critério (peso)	Onde está
Validação da feature com usuários (1,5)	Q1
Planejamento Agile com IA (2,0)	Q2
Avaliação crítica do planejamento (2,0)	Q3 + Q3A
Qualidade técnica da implementação (2,0)	Q4 + Q7 (app rodando)
Avaliação arquitetural baseada em SOLID (1,5)	Q5
Refatoração e análise crítica das IAs (1,0)	Q6